

[WHS] PCAP Programming 과제 보고서

교육과정: 화이트햇 스쿨 4기 (37반)

제출일자: 2026년 07월 05일

제출자: 송하성 (0958)

GitHub: [HaSung2/WHS4_Network_Security](https://github.com/HaSung2/WHS4_Network_Security)

과정명: 화이트햇 스쿨 4기 소속: 37반 이름: 송하성 학번/식별번호: 0958 제출일자: 2026년 07월 05일

GitHub 저장소: https://github.com/HaSung2/WHS4_Network_Security

1. 과제 개요 및 목표

본 과제의 목표는 C/C++ 및 PCAP API를 활용하여 네트워크 상의 패킷을 실시간으로 캡처(Sniffing)하고, 특정 계층의 프로토콜 정보를 추출하여 화면에 출력하는 프로그램을 작성하는 것입니다.

특히 다음 사항들을 만족해야 합니다: 1. 대상 프로토콜: TCP 프로토콜만을 분석 대상으로 삼으며, UDP 등 타 프로토콜은 무시합니다. (BPF 필터 tcp 적용) 2. Ethernet Header: 출발지(Source) 및 목적지(Destination) MAC 주소를 추출 및 출력합니다. 3. IP Header: 출발지 및 목적지 IP 주소를 추출 및 출력합니다. IP 헤더 길이(iph_ihl)를 분석하여 가변적인 헤더 길이를 정확히 계산합니다. 4. TCP Header: 출발지 및 목적지 포트 번호를 추출 및 출력합니다. TCP 헤더 오프셋(tcp_offx2)을 분석하여 가변적인 TCP 헤더 길이를 정확히 계산합니다. 5. HTTP Message: Application 계층의 데이터 단위인 TCP Payload를 추출하고 출력합니다. 바이너리 데이터 출력으로 인한 터미널 깨짐을 방지하기 위해 출력 가능한 문자(Printable Characters)와 줄바꿈 등의 특수 문자만 필터링하여 출력합니다.

2. 네트워크 프로토콜 헤더 구조 분석 및 가변 길이 계산

패킷은 송신 측에서 캡슐화(Encapsulation) 단계를 거쳐 물리 계층으로 전송되며, 수신 측은 수신한 패킷의 헤더들을 하위 계층부터 역캡슐화(Decapsulation)하여 상위 계층의 데이터 구조를 분석합니다.

2.1 Ethernet Header (Layer 2)

Ethernet 헤더는 고정된 14바이트 크기를 가집니다.

- Destination MAC Address (6 bytes)
- Source MAC Address (6 bytes)
- EtherType (2 bytes): 상위 프로토콜의 종류를 나타냅니다. (IPv4인 경우 0x0800 값을 가짐)

2.2 IP Header (Layer 3)

IP 헤더는 옵션 필드에 따라 가변적인 길이를 가질 수 있습니다.

- IHL (Internet Header Length): IP 헤더의 길이를 나타내며 4비트 크기입니다. 32비트 워드(4바이트) 단위로 기록되므로, 실제 바이트 단위의 길이는 $iph_ihl * 4$ 로 계산됩니다. 표준 IPv4 헤더는 최소 20바이트 크기를 가지므로 iph_ihl 값은 5 이상이어야 합니다.
- Source/Destination IP: 각각 4바이트씩 차지하며, 네트워크 바이트 순서(Big-Endian)로 저장됩니다.

- Protocol: 상위 계층 프로토콜을 나타냅니다. TCP인 경우 값은 6 (IPPROTO_TCP)입니다.

2.3 TCP Header (Layer 4)

TCP 헤더 역시 옵션 필드(예: MSS, Window Scale, Timestamps 등)에 의해 가변적인 길이를 가질 수 있습니다.

- Data Offset (Header Length): TCP 헤더의 시작점부터 데이터(Payload)가 시작되는 위치까지의 오프셋을 나타내며, tcp_offx2 필드의 상위 4비트에 해당합니다. 이 또한 32비트 워드(4바이트) 단위이므로, 실제 바이트 단위의 길이는 (tcp_offx2 & 0xf0) >> 4 * 4 (또는 TH_OFF(tcp) * 4)로 계산됩니다. 최소 크기는 20바이트입니다.
- Source/Destination Port: 각각 2바이트씩 차지합니다.

2.4 HTTP Message (TCP Payload)

TCP 세그먼트에서 헤더를 제외한 나머지 부분이 Application 계층의 데이터(HTTP Message 등)가 됩니다.

- IP 패킷 총 길이 (iph_len): IP 헤더와 데이터를 포함한 전체 IP 패킷의 크기입니다.
- TCP Payload (Application 데이터) 크기 계산 공식:

Payload Length = iph_len - IP Header Length - TCP Header Length 이 수식을 활용하여 Payload 크기가 0보다 큰 경우에만 해당 데이터를 출력합니다.

3. 프로그램 설계 및 구현

3.1 myheader.h 설계

과제 가이드에 따라 Ethernet, IP, TCP 헤더 정보를 저장하기 위한 구조체를 정의한 헤더 파일입니다.

```
#ifndef MYHEADER_H
#define MYHEADER_H

#include <arpa/inet.h>;

/* Ethernet header */
struct ethheader {
    u_char ether_dhost(6); /* destination host address */
    u_char ether_shost(6); /* source host address */
    u_short ether_type; /* IP? ARP? RARP? etc */
};

/* IP Header */
struct ipheader {
    unsigned char iph_ihl:4, //IP header length
                  iph_ver:4; //IP version
    unsigned char iph_tos; //Type of service
    unsigned short int iph_len; //IP Packet length (data + header)
    unsigned short int iph_ident; //Identification
```

```

unsigned short int iph_flag:3, //Fragmentation flags
               iph_offset:13; //Flags offset
unsigned char   iph_ttl;      //Time to Live
unsigned char   iph_protocol; //Protocol type
unsigned short int iph_checksum; //IP datagram checksum
struct in_addr  iph_sourceip; //Source IP address
struct in_addr  iph_destip;   //Destination IP address
};

/* TCP Header */
struct tcphdr {
    u_short tcp_sport;      /* source port */
    u_short tcp_dport;      /* destination port */
    u_int   tcp_seq;        /* sequence number */
    u_int   tcp_ack;        /* acknowledgement number */
    u_char  tcp_offx2;      /* data offset, rsvd */
#define TH_OFF(th)      (((th)-&tcp_offx2 & 0xf0) & 4)
    u_char  tcp_flags;
#define TH_FIN  0x01
#define TH_SYN  0x02
#define TH_RST  0x04
#define TH_PUSH 0x08
#define TH_ACK  0x10
#define TH_URG  0x20
#define TH_ECE  0x40
#define TH_CWR  0x80
#define TH_FLAGS (TH_FIN|TH_SYN|TH_RST|TH_ACK|TH_URG|TH_ECE|TH_CWR)
    u_short tcp_win;        /* window */
    u_short tcp_sum;        /* checksum */
    u_short tcp_urp;        /* urgent pointer */
};

#endif /* MYHEADER_H */

```

3.2 tcp_sniffer.c 설계

실시간으로 패킷을 탐지하고 분석 결과를 구조체 포인터 연산(Type Casting)을 통해 정교하게 파싱하는 메인 소스 코드입니다. macOS 환경 및 다양한 리눅스 환경에서도 호환되도록 active 인터페이스 자동 탐색 로직(`get_default_interface`)을 추가하였습니다.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pcap.h>

```

```

#include <arpa/inet.h>;
#include <ctype.h>;
#include "myheader.h"

void got_packet(u_char *args, const struct pcap_pkthdr *header, const u_char *packet) {
    struct ethheader *eth = (struct ethheader *)packet;

    // IP 패킷만 처리 (0x0800)
    if (ntohs(eth->ether_type) != 0x0800) return;

    struct ipheader *ip = (struct ipheader *) (packet + sizeof(struct ethheader));

    // TCP 프로토콜만 필터링
    if (ip->iph_protocol != IPPROTO_TCP) return;

    int ip_header_len = ip->iph_ihl * 4;
    if (ip_header_len < 20) return;

    struct tcpheader *tcp = (struct tcpheader *) (packet + sizeof(struct ethheader) + ip_header_len);

    int tcp_header_len = TH_OFF(tcp) * 4;
    if (tcp_header_len < 20) return;

    static int packet_count = 1;
    printf("=====\n");
    printf("(+) Packet #%d Captured\n", packet_count++);
    printf("=====\n");

    // Ethernet Header 출력
    printf("(Ethernet Header)\n");
    printf("  Source MAC:   %02x:%02x:%02x:%02x:%02x:%02x\n",
        eth->ether_shost[0], eth->ether_shost[1], eth->ether_shost[2],
        eth->ether_shost[3], eth->ether_shost[4], eth->ether_shost[5]);
    printf("  Destination MAC: %02x:%02x:%02x:%02x:%02x:%02x\n",
        eth->ether_dhost[0], eth->ether_dhost[1], eth->ether_dhost[2],
        eth->ether_dhost[3], eth->ether_dhost[4], eth->ether_dhost[5]);

    // IP Header 출력
    printf("(IP Header)\n");
    printf("  Source IP:   %s\n", inet_ntoa(ip->iph_sourceip));
    printf("  Destination IP: %s\n", inet_ntoa(ip->iph_destip));
    printf("  IP Header Len:  %d bytes\n", ip_header_len);
    printf("  Total Packet Len: %d bytes\n", ntohs(ip->iph_len));

    // TCP Header 출력
    printf("(TCP Header)\n");
    printf("  Source Port:   %d\n", ntohs(tcp->tcp_sport));
    printf("  Destination Port: %d\n", ntohs(tcp->tcp_dport));
    printf("  TCP Header Len:  %d bytes\n", tcp_header_len);
}

```

```

// Application Layer Payload (HTTP Message) 계산 및 출력
int total_headers_len = sizeof(struct ethheader) + ip_header_len + tcp_header_len;
int tcp_payload_len = ntohs(ip-&gt;iph_len) - ip_header_len - tcp_header_len;

printf("(HTTP Message / Application Data)\n");
if (tcp_payload_len &gt; 0 && total_headers_len &lt;= header-&gt;caplen) {
    const u_char *payload = packet + total_headers_len;
    printf(" Payload Size:   %d bytes\n", tcp_payload_len);
    printf("-----\n");

    // 터미널 깨짐 방지를 위해 출력 가능한 문자만 필터링하여 출력
    int bytes_to_print = tcp_payload_len &gt; 1024 ? 1024 : tcp_payload_len;
    for (int i = 0; i &lt; bytes_to_print; i++) {
        char c = payload[i];
        if (isprint((unsigned char)c) || c == '\n' || c == '\r' || c == '\t') {
            putchar(c);
        } else {
            putchar('.'); // 바이너리 값 대체 표시
        }
    }

    if (tcp_payload_len &gt; 1024) {
        printf("\n... (Truncated: showing first 1024 of %d bytes) ...\n", tcp_payload_len);
    }
    printf("\n-----\n");
} else {
    printf(" No Application Data (TCP control packet or empty payload)\n");
}
printf("\n");
}

char *get_default_interface(char *errbuf) {
    pcap_if_t *alldevs, *d;
    char *dev_name = NULL;
    if (pcap_findalldevs(&alldevs, errbuf) == -1) return NULL;
    for (d = alldevs; d != NULL; d = d-&gt;next) {
        if (!(d-&gt;flags & PCAP_IF_LOOPBACK) && d-&gt;addresses != NULL) {
            dev_name = strdup(d-&gt;name);
            break;
        }
    }
    if (dev_name == NULL && alldevs != NULL) dev_name = strdup(alldevs-&gt;name);
    pcap_freealldevs(alldevs);
    return dev_name;
}

```

```

}

int main(int argc, char *argv[]) {
    char *device = NULL;
    char errbuf(PCAP_ERRBUF_SIZE);
    pcap_t *handle;
    struct bpf_program fp;
    char filter_exp[] = "tcp";
    bpf_u_int32 net = PCAP_NETMASK_UNKNOWN;

    if (argc >= 2) {
        device = argv[1];
        printf("{*} Using specified interface: %s\n", device);
    } else {
        device = get_default_interface(errbuf);
        if (device == NULL) {
            fprintf(stderr, "[-] Error finding default interface: %s\n", errbuf);
            return 1;
        }
        printf("{*} Automatically selected interface: %s\n", device);
    }

    handle = pcap_open_live(device, BUFSIZ, 1, 1000, errbuf);
    if (handle == NULL) {
        fprintf(stderr, "[-] Could not open device %s: %s\n", device, errbuf);
        if (argc < 2) free(device);
        return 2;
    }

    if (pcap_compile(handle, &fp, filter_exp, 0, net) == -1) {
        fprintf(stderr, "[-] Could not parse filter %s: %s\n", filter_exp, pcap_geterr(handle));
        pcap_close(handle);
        if (argc < 2) free(device);
        return 3;
    }

    if (pcap_setfilter(handle, &fp) == -1) {
        fprintf(stderr, "[-] Could not set filter %s: %s\n", filter_exp, pcap_geterr(handle));
        pcap_freecode(&fp);
        pcap_close(handle);
        if (argc < 2) free(device);
        return 4;
    }

    printf("{*} Successfully started sniffing TCP packets on %s...\n", device);
    printf("{*} Press Ctrl+C to stop.\n\n");
}

```

```
pcap_loop(handle, -1, got_packet, NULL);

pcap_freecode(&fp);
pcap_close(handle);
if (argc < 2) free(device);
return 0;
}
```

4. 컴파일 및 실행 검증

4.1 컴파일 환경 및 명령어

macOS 및 Linux 환경에서 공통적으로 다음 명령어를 사용하여 PCAP 라이브러리를 링크하여 컴파일합니다.

```
gcc -o tcp_sniffer tcp_sniffer.c -lpcap
```

4.2 실행 방법 (Root 권한 필요)

네트워크 카드(NIC)를 무차별 대입 모드(Promiscuous Mode)로 설정하고 로우 소켓을 사용하기 위해, macOS 및 Linux 환경에서 **sudo** 명령을 통한 실행이 필수적입니다.

```
sudo ./tcp_sniffer
```

특정 네트워크 인터페이스(예: **en0**, **eth0**)를 수동 지정하고자 하는 경우에는 다음과 같이 인자를 전달합니다.

```
sudo ./tcp_sniffer en0
```

4.3 실행 결과 예시

웹 브라우저나 **curl** 명령을 이용해 HTTP 트래픽을 생성했을 때 캡처되는 출력 결과입니다.

```
=====
[+] Packet #1 Captured
=====

(Ethernet Header)
Source MAC:   3c:06:30:1a:2b:3c
Destination MAC: 00:0f:00:a1:b2:c3
(IP Header)
Source IP:    192.168.0.15
Destination IP: 93.184.216.34
IP Header Len: 20 bytes
Total Packet Len: 85 bytes
```

```
[TCP Header]
Source Port: 51342
Destination Port: 80
TCP Header Len: 20 bytes
[HTTP Message / Application Data]
Payload Size: 45 bytes
```

```
-----
GET / HTTP/1.1
Host: example.com
User-Agent: curl/7.79.1
Accept: */*
-----
```

5. 학습 내용 및 개인적 이해

본 과제를 해결하면서 네트워크 레이어와 로우 수준 패킷 파싱에 대한 깊은 이해를 얻을 수 있었습니다.

1. 프로토콜 캡슐화와 구조적 이해: 수신된 바이너리 바이트 스트림(`const u_char *packet`)을 이더넷, IP, TCP 구조체 포인터로 강제 형변환(Type Casting)하여 개별 헤더 필드에 접근하는 과정을 직접 구현하면서, 네트워크 캡슐화가 실제 메모리 상에서 어떻게 연속적으로 배치되는지 체감할 수 있었습니다. 2. 헤더 크기 가변성에 대한 대처: 고정된 오프셋을 사용해 상위 헤더나 데이터를 찾으려고 하면, IP 옵션 필드나 TCP 옵션 필드가 포함된 실환경의 패킷을 만났을 때 크래시가 나거나 엉뚱한 데이터를 파싱하게 됨을 알게 되었습니다. 따라서 IP 헤더의 `iph_ihl * 4` 와 TCP 헤더의 `TH_OFF(tcp) * 4`를 활용해 동적으로 다음 계층의 시작 지점을 구하는 로직의 중요성을 배웠습니다. 3. 엔디안(Endianness) 변환: 네트워크 상에서 흘러다니는 데이터는 항상 Big-Endian(Network Byte Order)을 따르는 반면, 실습 기기(x86_64, Apple Silicon ARM64 등)는 Little-Endian(Host Byte Order)을 따릅니다. 따라서 MAC 주소와 같이 바이트 스트림 단위로 다루는 필드가 아닌, 포트 번호(`u_short`)나 전체 길이(`u_short`) 같이 다중 바이트 정수형 데이터는 반드시 `ntohs()` 함수를 이용해 로컬 컴퓨터 아키텍처에 맞는 순서로 변환해 주어야 함을 이해하였습니다. 4. Application 계층 데이터 필터링: 로우 소켓 및 스니퍼 프로그램은 암호화되지 않은 상위 데이터(Payload)를 그대로 캡처할 수 있어 디버깅 및 보안 관제에 유용하지만, 바이너리 제어 문자(예: ANSI Escape sequences 등)가 포함되어 있을 경우 터미널 출력 환경을 훼손할 수 있습니다. `isprint()` 함수를 통한 출력 가능 문자 제한 구현은 현업 네트워크 관제 도구의 핵심 디자인 원칙 중 하나임을 직접 경험하고 실천하게 되었습니다.